

Practical Experiment: Compare the response speed of open-source Large Language Models (LLMs) to find the fastest model for real-time applications

Meeting 8 Provision a Server for OllamaModelTester Practical Experiment

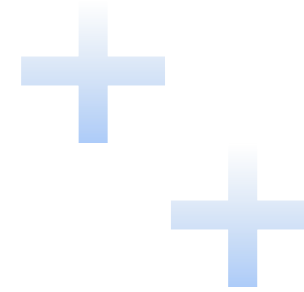
Presented by: Mariyan Vasilev Apostolov
Date: 15th of September 2026
Version: 1.0

Agenda

- Introduction and Objectives
- Phase One – Infrastructure Provisioning
- Phase Two – Environment Configuration
- Kubernetes and Dependencies
- Phase Three – Deployment
- Validation Protocol
- Phase Four – Kubernetes Dashboard
- Architecture Summary



Introduction and Objectives



Purpose of the Experiment

The OllamaModelTester requires a reliable, scalable, and accessible server environment to function effectively. Traditional local development setups often lack the computational resources and flexibility needed for testing Large Language Models.

By leveraging cloud infrastructure and container orchestration, we create an environment that is reproducible, isolated, and capable of handling the demands of AI model testing.

Primary Objectives:

- Establish a cloud-based server on Google Cloud Platform
- Configure remote access and container management capabilities
- Deploy and validate a Kubernetes-based orchestration environment
- Provide graphical monitoring through the Kubernetes Dashboard

This experiment serves as the foundational layer for subsequent testing and development activities.

Phase One – Infrastructure Provisioning

Creating the VM on Google Cloud Platform

The first phase involved establishing foundational compute resources on GCP.

Project Setup:

- Created a new project within the GCP Console
- Enabled the Compute Engine API for virtual machine services

Virtual Machine Configuration:

- Selected Ubuntu 24.04 LTS for stability and modern package availability
- Chose e2-medium instance: 2 vCPUs, 4 GB Memory
- This specification balances Kubernetes overhead with LLM workload requirements

Network and Security:

- Configured VPC network settings for necessary traffic
- Created firewall rules for HTTP, HTTPS, and RDP (port 3389)
- Generated SSH keys for secure administrative access

This dual-access approach provides both command-line and graphical management capabilities.

Phase Two – Environment Configuration

Graphical Interface and Containerization



XFCE4 and XRDP Configuration:

- Selected XFCE4 as a lightweight desktop environment preserving resources for AI workloads
- Installed XRDP to enable Remote Desktop Protocol connections
- This allows graphical access to Kubernetes Dashboard and browser-based testing tools
- Unlike heavier alternatives like GNOME or KDE, XFCE4 consumes minimal system resources

Docker Installation:

- Installed Docker Engine and Command Line Interface
- Configured non-root user permissions for improved security
- Docker serves as the containerization layer for isolated application deployment
- Functions as the container runtime for Minikube



Kubernetes and Dependencies

Orchestration Setup

Minikube and Kubernetes Tools:

- Minikube provides a single-node Kubernetes cluster within a virtual machine
- This approach avoids overhead of production-grade multi-node clusters
- Installed kubectl for cluster interaction and management
- kubeadm provides cluster bootstrapping capabilities when needed

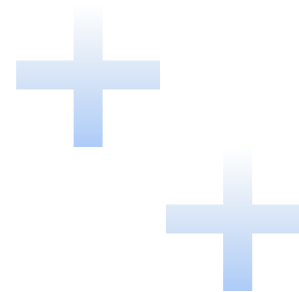
Required Dependencies:

- conntrack for network connection tracking (essential for Kubernetes networking)
- crictl for Container Runtime Interface operations
- Additional packages for container networking, storage management, and system monitoring
- Each dependency verified for compatibility with Ubuntu 24.04 and Kubernetes version

Nginx Installation:

- Serves as reverse proxy and potential ingress controller
- Provides flexibility to expose applications while maintaining security
- Capable of load balancing and routing traffic to backend services

Phase Three – Deployment



Initializing Cluster and Deploying Application

Cluster Initialization:

- Started Minikube with resource allocations matching VM specifications
- Verified control plane status: API server, scheduler, controller manager, and etcd
- Confirmed healthy cluster ready to accept workload deployments

Application Deployment Strategy:

- Created deployment manifests defining desired state for OllamaModelTester
- Deployments manage ReplicaSets, which in turn manage Pods
- Configured container image, resource requirements, environment variables, and health checks
- Created Services to expose application: ClusterIP for internal, NodePort for external access

This hierarchical structure provides capabilities for rolling updates, rollbacks, and scaling operations.

8 Validation Protocol

Verifying System Integrity

Pod Validation:

- Confirmed all Pods achieved the Running state
- Verified no unexpected restarts indicating stability
- Examined logs to identify runtime errors or warnings
- Absence of critical errors confirmed application started correctly

Service Validation:

- Confirmed endpoint lists contain expected Pod IP addresses
- Tested connectivity by sending requests to service endpoints
- Verified internal cluster communication and external access configuration

Deployment Validation:

- Confirmed desired replica count matches available replicas
- Examined deployment status conditions for successful rollout
- Ensured Kubernetes properly manages application lifecycle

This comprehensive validation provides confidence that the environment functions as intended.

Phase Four – Kubernetes Dashboard

Visualizing Cluster State

Dashboard Deployment:

- Deployed using official Kubernetes manifest
- Created service account with appropriate permissions
- Generated bearer token for secure authentication

Dashboard Capabilities:

- Displays CPU and memory utilization across the cluster
- Shows Pod status, replica counts, and service endpoints
- Provides log viewing and event inspection for troubleshooting
- Enables visual monitoring without command-line queries

Navigation:

- Accessed namespace containing OllamaModelTester deployment
- Inspected application status through intuitive graphical interface
- Simplified identification of potential issues and resource consumption

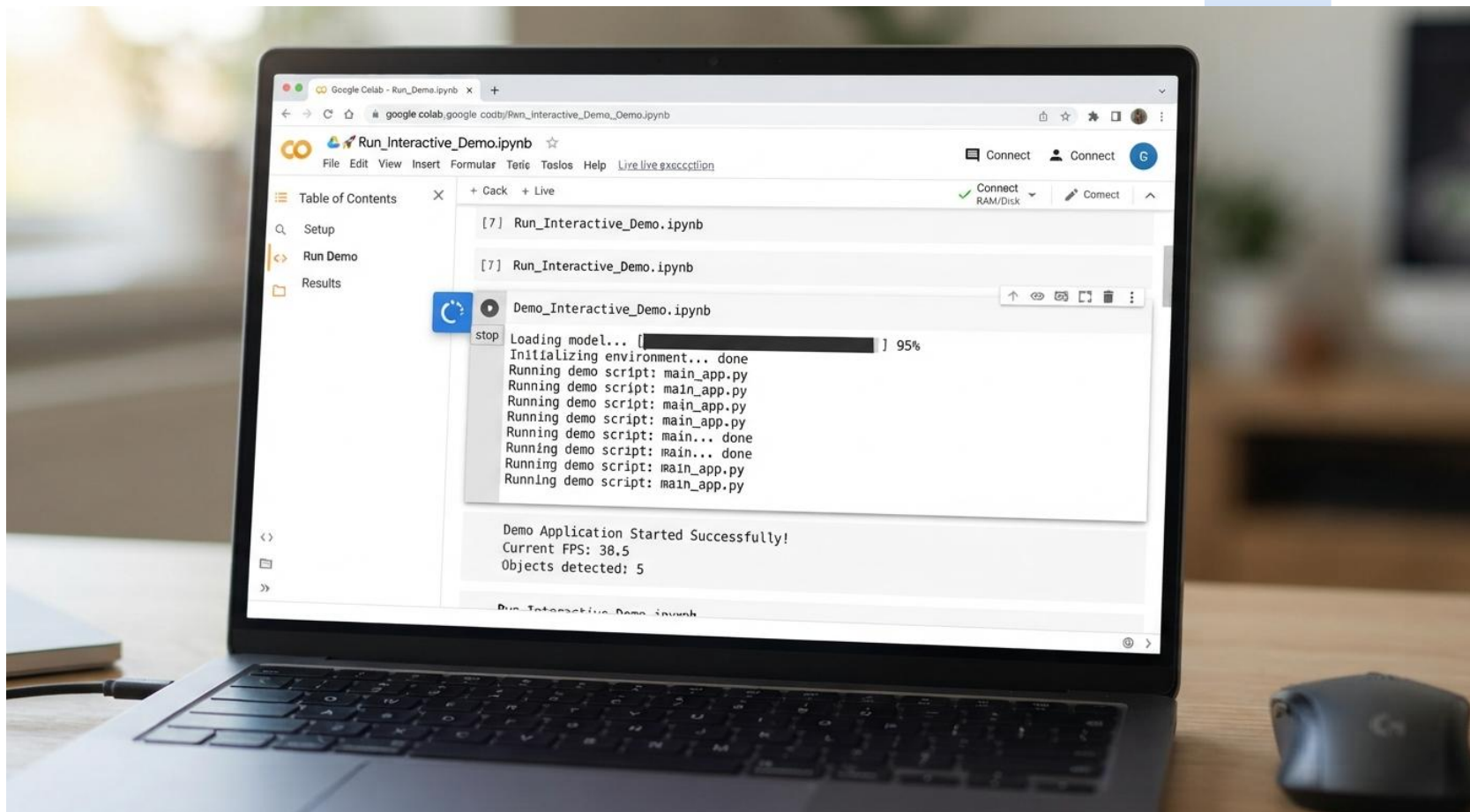
10

Architecture Summary

Layer	Components
Infrastructure	GCP VM, Ubuntu 24.04 LTS
Access	XRDP, XFCE4 Desktop
Containerization	Docker Engine
Orchestration	Minikube, Kubernetes
Application	OllamaModelTester Pods, Services
Web Server	Nginx
Monitoring	Kubernetes Dashboard

11

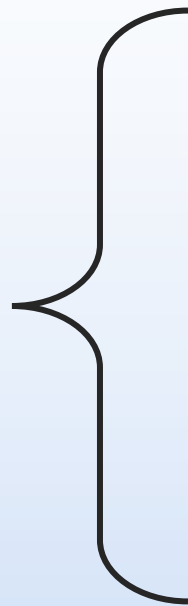
LIVE DEMO



References

- Install XRDP on Ubuntu – <https://www.cherrysevers.com/blog/install-xrdp-on-ubuntu-2404>
- Install Docker Engine – <https://docs.docker.com/engine/install/ubuntu/>
- Install Kubernetes – <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/>
- Install Minikube – <https://minikube.sigs.k8s.io/docs/start/?arch=%2Flinux%2Fx86-64%2Fstable%2Fbinary+download>
- Install Chrome – <https://oneuptime.com/blog/post/2026-03-02-how-to-install-google-chrome-on-ubuntu/view>
- Kubernetes basics – <https://kubernetes.io/docs/tutorials/kubernetes-basics/>
- Start Minikube – <https://minikube.sigs.k8s.io/docs/drivers/docker/>
- Minikube Dashboard – <https://minikube.sigs.k8s.io/docs/handbook/dashboard/>
- Minikube Dashboard commands – <https://minikube.sigs.k8s.io/docs/commands/dashboard/>
- Install Git – <https://git-scm.com/install/linux>
- Getting started with Git – <https://git-scm.com/book/ms/v2/Getting-Started-First-Time-Git-Setup>
- Install Nginx – <https://ubuntu.com/tutorials/install-and-configure-nginx#2-installing-nginx>

Thank you



Mariyan Vasilev Apostolov

mariyan.apostolov89@gmail.com